

task_bridge

Ваша таск-борда і ваш адзіны крыніца праўды — ідэальна сінхранізаваныя ў абодва бакі.

Агульны агляд

task_bridge — гэта ўніверсальны, дэкапліраваны, двухбаковы рухавік сінхранізацыі таск/борд у Go. Ён падтрымлівае сінхранізацыю адзінай крыніцы праўды (SSoT) праекта па рабочых элементах (SQLite) з яго дакументацыяй у трэкеры і аддаленай бордай (першыя цэль — ClickUp; плануецца Jira/Linear), выкарыстоўваючы дэтэрмінісцкую логіку «перамагае апошняе змена», спачатку «сухая праверка» і семантыку без страт і пашкоджанняў.

Кароткае апісанне

Праектна-незалежны падмадуль Go, які двухбакова сінхранізуе SSoT рабочых элементаў SQLite ↔ дакументацыю ў трэкеры ↔ аддаленую борду (спачатку ClickUp). Дэтэрмінісцкая логіка «перамагае апошняе змена», спачатку «сухая праверка», вебхукі з праверкай HMAC; усе крэды і ідэнтыфікатары ўводзяцца спажываемым падчас выканання.

Падрабязнае апісанне

Раней ці позна кожная каманда вядзе дзве кнігі адной і той жа працы: сапраўдную — код, дакументы, унутраную базу даных — і ту, за якой сочаць менеджары, борду нахштальт ClickUp. Яны разыходзяцца ў момант, калі хоць адзін бок змяняецца, а ручная ўзгадненне — гэта менавіта той нудны, схільны да памылак занятак, які ніхто не робіць рэгулярна. task_bridge створаны, каб знішчыць гэтую прабелу, разглядаючы ўсе тры прадстаўленні як адзіную сістэму, якая павінна заставацца ў поўнай сінхранізацыі: **адзіная крыніца праўды па рабочых элементах праекта (SQLite)**, яго дакументацыю ў **трэкеры і аддаленую борду** — першай падтрымліваецца ClickUp, а Jira і Linear плануецца ў будучыні. Сінхранізацыя

дэтэрмінісцкая («перамагае апошняя змена»), спачатку праходзіць «сухую праверку» і спраектавана вакол адзінай нязменнай абяцанкі: яна ніколі не пашкодзіць і не страціць даныя і ніколі не пакіне адзін бок састарэлым. У сферы, дзе неасцярожная сінхранізацыя можа знішчыць тыдзень працы, менавіта гэтая бяспека і ёсць галоўны сэнс. Архітэктурна гэта строгі падмадуль, які спажываецца іншымі праектамі і цалкам праектна-незалежны згодна з кантрактам дэкаплінгу канстытуцыі (§11.4.28): ён не ўключае ніякіх праектна-спецыфічных значэнняў, а ўсе крэды, ідэнтыфікатары борд/папкі, палі ключоў элементаў і шляхі да баз даных уводзяцца спажывуцом падчас выканання праз `pkg/config.Config`. Модуль пабудаваны па ўзроўнях: CLI (`reconcile/push/pull/resolve/status/conflicts/init`) і доўгажывучы дэман (прымальнік вебхукаў + cron-узгадненне); тонкая абгортка кліента над MIT-ліцэнзаваным `raksul/go-clickup`; рэзальвер, які пераўтварае URL борд/папкі ў ідэнтыфікатары праз жывыя праверкі API (без здагадак па граматыцы URL); мапер паміж лакальнымі рабочымі элементамі і палямі аддаленых задач; рухавік сінхранізацыі з логікай «перамагае апошняя змена» і вырашэннем канфліктаў; прымальнік вебхукаў з праверкай X-Signature HMAC-SHA256. Ён адкрыта гаворыць пра сваю сталасць: гэта P1-каркас — разметка, інтэрфейсы, кропкі ўваходу і мяжа дэкаплінгу на месцы, але логіка сінхранізацыі і жывыя запыты да ClickUp яшчэ не рэалізаваны (кожны заглушка вяртае відавочную памылку «не рэалізавана» згодна з правілам «без фэйкаў»).

Чаму мы яго стварылі

Каманды захоўваюць «сапраўдны» стан працы ў кодзе/дакументах, а менеджары жывуць на бордзе нахштальт ClickUp — і яны пастаянна разыходзяцца. `task_bridge` робіць іх адзінай сістэмай, сінхранізуючы дэтэрмінісцка і бяспечна, каб ні адзін бок не састарэў і не стаў няправільным.

Змест

Чаму гэта змяняе правілы гульні

Двухбаковая сінхранізацыя дошак звычайна з’яўляецца аднаразовай, жорстка прывязанай інтэграцыяй, якую кожная каманда дрэнна перапісвае нанова. `task_bridge` пераасэнсоўвае яе як паўторна выкарыстоўваемую бібліятэку з ін’екцыяй аўтэнтыфікацыйных дадзеных і строгімі гарантыямі бяспекі даных — спачатку тэст-рэжым, дэтэрмінісцкае правіла апошняй змены, падпісаныя HMAC-паведамленні — каб любы

праект мог атрымаць надзейную інтэграцыю з дошкамі праз ін'екцыю канфігурацыі, а не праз напісанне чарговага крохкага канэктара, звязанага з унутранай архітэктурай.

Што новага

- Трохбаковая двухбаковая сінхранізацыя: SQLite SSoT ↔ дакументы трэкера ↔ аддаленая дошка.
- Поўная дэкаплінгаванасць (§11.4.28): ніякіх праектных зменных; усе дадзеныя ін'ектуюцца падчас выканання.
- Развязванне URL→ID у рэжыме Live-API замест крохкага парсінгу URL-граматыкі.
- Падпісаныя HMAC-SHA256 вэбхукі для прыёму падзей у рэжыме рэальнага часу.

Праблемы і рашэнні

- **Бяспека даных паміж трыма крыніцамі:** вырашана праз дэтэрмінісцкае правіла апошняй змены, тэст-рэжым і відавочныя вынікі канфліктаў.
- **Паўторнае выкарыстанне без звязвання:** вырашана праз мяжу ін'екцыі pkg/config (няма праектных дэталей у пастаўцы).
- **Надзейная ідэнтыфікацыя дошак:** вырашана праз пераўтварэнне URL у ID з дапамогай жывых API-прабаў.
- **Часнасць каркаса:** вырашана праз вяртанне відавочных памылак “не рэалізавана” ў заглушках (няма імітацый).

Тэхнічны стэк (чаму + як)

- **Go** — рухавік, CLI (cmd/task_bridge) і дэман (cmd/task_bridged).
- **SQLite** — адзіная крыніца праўды для працаздольных элементаў.
- **raksul/go-clickup (MIT)** — абгортка транспарту ClickUp.
- **HMAC-SHA256** — праверка подпісу вэбхукаў.
- **cron + вэбхукі** — рэкансіляцыя дэмана + прыём падзей у рэжыме рэальнага часу.
- **pkg/config** — мяжа ін'екцыі аўтэнтыфікацыйных дадзеных/ID падчас выканання.

Часнасць статусу: гэта **P1-каркас** — логіка сінхранізацыі яшчэ не рэалізавана. Не падавайце як гатовы прадукт.